# Solution Architecture

iMovies MERN Project • Phase-wise Documentation

**Project:** iMovies MERN  **Document:** Solution Architecture

## 1. Architecture overview

iMovies follows a client–server architecture. React renders the user interface; the Express API validates requests and applies business rules; MongoDB stores persistent records. Authentication middleware protects user and admin endpoints.

## 2. Logical architecture

| Layer | Components | Responsibilities |
|---|---|---|
| Presentation | React/Vite, components, CSS | Movie list, forms, seat selection, booking pages, admin screens. |
| API / application | Node.js, Express routes | Request handling, validation, authorization and business logic. |
| Security | JWT middleware, bcrypt, role checks | Identity verification, password hashing and protected endpoints. |
| Persistence | MongoDB, Mongoose models | Users, movies, bookings and seat-reservation records. |

## 3. Request flow

Browser → React UI → HTTP request → Express route → authentication/validation → Mongoose model → MongoDB → JSON response → React UI update.

## 4. Main data entities

| Entity | Key fields (conceptual) | Purpose |
|---|---|---|
| User | name, email, password hash, role | Account and authorization. |
| Movie | title, description, genre, duration, rating, price | Movie catalogue. |
| Booking | userId, movieId, show date/time, seats, total amount, status | Ticket reservation and history. |
| SeatReservation | movie/show identity, seat identity, booking reference | Avoid overlapping seat reservations. |

## 5. API areas

| API area | Typical purpose |
|---|---|
| /api/health | Health check. |
| /api/register and /api/login | User account access (exact paths depend on server.js). |
| /api/movies | Read movie catalogue. |
| /api/bookings | Create and list a user’s bookings; cancellation route. |
| /api/admin/* | Admin login, movie management and booking review. |

## 6. Security boundaries

- Client-side controls improve usability but are not security boundaries.
- Server middleware must verify tokens and roles.
- Booking reads/deletes must be scoped to the authenticated user or explicitly authorized admin.
- Environment secrets must remain outside GitHub.
- Database credentials and production CORS settings must be configured for deployment.

## 7. Deployment view

Development: browser on localhost:5174 → API on localhost:5000 → local MongoDB. Production deployment requires a hosted frontend, hosted backend, managed database, environment variables, HTTPS, and correct CORS/API base URL settings.

## 8. Architecture caveat

This document describes the intended logical architecture based on the current project context. Before final submission, compare endpoint names and model fields with the actual current source code.

**Prepared for academic project documentation.** Verify implementation-specific details and fill in actual test evidence before final submission.